Trend

1. cloneing git repo

```
ubuntu@ip-172-31-4-49:~$ git clone https://github.com/Vennilavanguvi/Trend.git
Cloning into 'Trend'...
remote: Enumerating objects: 80, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 80 (delta 0), reused 0 (delta 0), pack-reused 79 (from 1)
Receiving objects: 100% (80/80), 8.58 MiB | 11.64 MiB/s, done.
Resolving deltas: 100% (1/1), done.
ubuntu@ip-172-31-4-49:~$
```

2. creating dockerfile.

```
# Use nginx web server
FROM nginx:alpine

# Remove default nginx website
RUN rm -rf /usr/share/nginx/html/*

# Copy production build
COPY dist/ /usr/share/nginx/html/

# Expose port 80 inside container
EXPOSE 80
~
~
~
~
~
~
```

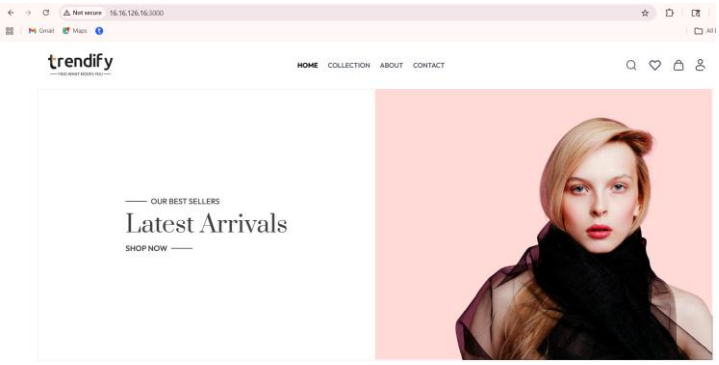
3. build docker image

```
ubuntu@ip-172-31-4-49:~/Trend$ vi Dockerfile
ubuntu@ip-172-31-4-49:~/Trend$ docker build -t trend-app .
[+] Building 2.0s (9/9) FINISHED                                  docker:default
 => [internal] load build definition from Dockerfile                        0.0s
 => => transferring dockerfile: 251B                                        0.0s
 => [internal] load metadata for docker.io/library/nginx:alpine             0.7s
 => [auth] library/nginx:pull token for registry-1.docker.io                0.0s
 => [internal] load .dockerignore                                           0.0s
 => => transferring context: 2B                                             0.0s
 => [1/3] FROM docker.io/library/nginx:alpine@sha256:582c496ccf79d8aa6f82   0.1s
 => => resolve docker.io/library/nginx:alpine@sha256:582c496ccf79d8aa6f82   0.1s
 => [internal] load build context                                           0.2s
 => => transferring context: 9.24MB                                         0.1s
 => CACHED [2/3] RUN rm -rf /usr/share/nginx/html/*                         0.0s
 => [3/3] COPY dist/ /usr/share/nginx/html/                                 0.1s
 => exporting to image                                                      0.8s
 => => exporting layers                                                     0.5s
 => => exporting manifest sha256:a7566f559a2112b6ed604aa307d9e40886575114   0.0s
 => => exporting config sha256:8c095b62c2af63e169774683a68935c3e51d02fb0c   0.0s
 => => exporting attestation manifest sha256:cf426c2ed13abdf1973632f9eeed   0.0s
 => => exporting manifest list sha256:d24448914e6fac9aaf66a8841d2fd263ef9   0.0s
 => => naming to docker.io/library/trend-app:latest                         0.0s
 => => unpacking to docker.io/library/trend-app:latest                      0.1s
ubuntu@ip-172-31-4-49:~/Trend$ docker run -d -p 3000:80 trend-app
483afe9ca2bc462f0ce0ecffcbceb880ef7c509d547a88c3b07948282254ad5e
ubuntu@ip-172-31-4-49:~/Trend$
```

4. push to docker hub

```
ubuntu@ip-172-31-4-49:~/Trend$ docker tag trend-app dugyalaravali28/trend-app
ubuntu@ip-172-31-4-49:~/Trend$ docker push dugyalaravali28/trend-app
Using default tag: latest
The push refers to repository [docker.io/dugyalaravali28/trend-app]
a71873b303e8: Mounted from dugyalaravali28/vite-app
34dfdd2ef1f9: Mounted from dugyalaravali28/vite-app
c8a2fa3a88d2: Mounted from dugyalaravali28/vite-app
69a66bce64d2: Mounted from dugyalaravali28/vite-app
fb08f5030179: Pushed
589002ba0eae: Mounted from dugyalaravali28/vite-app
ff9f59a6a62e: Mounted from dugyalaravali28/vite-app
1165b869c51a: Mounted from dugyalaravali28/vite-app
f03becc8ac15: Mounted from dugyalaravali28/vite-app
70952b9bc60a: Pushed
15e759724ff6: Mounted from dugyalaravali28/vite-app
latest: digest: sha256:d24448914e6fac9aaf66a8841d2fd263ef9340f57bd454f93c2928fa14c818c9 size: 856
ubuntu@ip-172-31-4-49:~/Trend$
```

5. tested locally



6. Creating eks

```
ubuntu@ip-172-31-37-161:~$ eksctl create cluster --name my-cluster --region ap-s
outh-1
2026-04-16 06:31:09 [ℹ]  eksctl version 0.225.0
2026-04-16 06:31:09 [ℹ]  using region ap-south-1
2026-04-16 06:31:09 [ℹ]  setting availability zones to [ap-south-1c ap-south-1b
ap-south-1a]
2026-04-16 06:31:09 [ℹ]  subnets for ap-south-1c - public:192.168.0.0/19 private
:192.168.96.0/19
2026-04-16 06:31:09 [ℹ]  subnets for ap-south-1b - public:192.168.32.0/19 privat
e:192.168.128.0/19
2026-04-16 06:31:09 [ℹ]  subnets for ap-south-1a - public:192.168.64.0/19 privat
e:192.168.160.0/19
2026-04-16 06:31:09 [ℹ]  nodegroup "ng-41c5dceb" will use "" [AmazonLinux2023/1.
34]
2026-04-16 06:31:09 [!]  Auto Mode will be enabled by default in an upcoming rel
ease of eksctl. This means managed node groups and managed networking add-ons wi
ll no longer be created by default. To maintain current behavior, explicitly set
 'autoModeConfig.enabled: false' in your cluster configuration. Learn more: http
s://eksctl.io/usage/auto-mode/
2026-04-16 06:31:09 [ℹ]  using Kubernetes version 1.34
```

```
2026-04-16 06:39:18 [!]  recommended policies were found for "vpc-cni" addon, bu
t since OIDC is disabled on the cluster, eksctl cannot configure the requested p
ermissions; the recommended way to provide IAM permissions for "vpc-cni" addon i
s via pod identity associations; after addon creation is completed, add all reco
mmended policies to the config file, under `addon.PodIdentityAssociations`, and
run `eksctl update addon`
2026-04-16 06:39:18 [ℹ]  creating addon: vpc-cni
2026-04-16 06:39:19 [ℹ]  successfully created addon: vpc-cni
2026-04-16 06:39:19 [ℹ]  creating addon: kube-proxy
2026-04-16 06:39:19 [ℹ]  successfully created addon: kube-proxy
2026-04-16 06:39:20 [ℹ]  creating addon: coredns
2026-04-16 06:39:20 [ℹ]  successfully created addon: coredns


2026-04-16 06:41:24 [ℹ]  building managed nodegroup stack "eksctl-my-cluster-nod
egroup-ng-41c5dceb"
2026-04-16 06:41:24 [ℹ]  deploying stack "eksctl-my-cluster-nodegroup-ng-41c5dce
b"
```

7. **Creating deployment and service yaml files.**

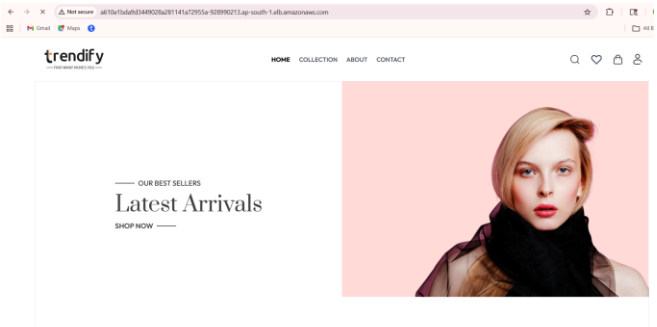
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: trend-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: trend-app
  template:
    metadata:
      labels:
        app: trend-app
    spec:
      containers:
      - name: trend-app
        image: yourdockerhubusername/trend-app
        ports:
        - containerPort: 80
~
~
~
~
  -- INSERT --
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: trend-service
spec:
  type: LoadBalancer
  selector:
    app: trend-app
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30000
~
~
~
~
~
~
~
```

8. Checking pods.

```
ubuntu@ip-172-31-37-161:~$ kubectl get nodes
NAME                                          STATUS   ROLES    AGE     V
ERSION
ip-192-168-25-187.ap-south-1.compute.internal   Ready    <none>   34m     v
1.34.6-eks-bbe087e
ip-192-168-54-134.ap-south-1.compute.internal   Ready    <none>   5m12s   v
1.34.6-eks-bbe087e
ip-192-168-64-105.ap-south-1.compute.internal   Ready    <none>   34m     v
1.34.6-eks-bbe087e
ubuntu@ip-172-31-37-161:~$ kubectl get svc
NAME            TYPE           CLUSTER-IP      EXTERNAL-IP
                                              PORT(S)        AGE
kubernetes      ClusterIP      10.100.0.1      <none>
                                              443/TCP        92m
trend-service   LoadBalancer   10.100.63.211   a610e1bda9d3449028a281141a72
955a-928990213.ap-south-1.elb.amazonaws.com   80:30000/TCP   29m
ubuntu@ip-172-31-37-161:~$
```

9. Tested locally.



10. Lunching ec2 server though terraform.

- **Provider.tf file**

```
ubuntu@ip-172-31-37-161: ~/Trend/terraform
provider "aws" {
  region = "ap-south-1"
}
~
```

- **Vpc.tf file**

```
ubuntu@ip-172-31-37-161: ~/Trend/terraform
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "public" {
  vpc_id     = aws_vpc.main.id
  cidr_block = "10.0.1.0/24"
  map_public_ip_on_launch = true
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id
}

resource "aws_route_table" "rt" {
  vpc_id = aws_vpc.main.id
}

resource "aws_route" "r" {
  route_table_id         = aws_route_table.rt.id
  destination_cidr_block = "0.0.0.0/0"
  gateway_id             = aws_internet_gateway.igw.id
}

resource "aws_route_table_association" "a" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.rt.id
}
~
~
~
~
~
```

- **Securitygroup.tf file**

ubuntu@ip-172-31-37-161: ~/Trend/terraform

```
resource "aws_security_group" "jenkins_sg" {
  vpc_id = aws_vpc.main.id

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 3000
    to_port     = 3000
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
~
~
~
~
~
```

- **Ec2.tf file**

ubuntu@ip-172-31-37-161: ~/Trend/terraform

```
resource "aws_instance" "jenkins" {
  ami           = "ami-0f5ee92e2d63afc18" # Ubuntu (ap-south-1)
  instance_type = "t3.micro"
  key_name      = "trends"

  subnet_id = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.jenkins_sg.id]

  user_data = <<-EOF
              #!/bin/bash
              apt update -y
              apt install -y openjdk-17-jdk docker.io git

              systemctl start docker
              systemctl enable docker

              # Jenkins install
              wget -O /usr/share/keyrings/jenkins-keyring.asc \
              https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key

              echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]" \
              https://pkg.jenkins.io/debian-stable binary/ > \
              /etc/apt/sources.list.d/jenkins.list

              apt update -y
              apt install -y jenkins

              systemctl start jenkins
              systemctl enable jenkins

              usermod -aG docker ubuntu
              usermod -aG docker jenkins
              EOF

  tags = {
    Name = "Jenkins-Server"
  }
}
~
~
~
~
~
```

- **Output.tf file**

ubuntu@ip-172-31-37-161: ~/Trend/terraform

```
output "jenkins_public_ip" {
  value = aws_instance.jenkins.public_ip
}
~
~
~
~
```

```
ubuntu@ip-172-31-37-161:~/Trend/terraform$ ls
ec2.tf  outputs.tf  provider.tf  securitygroup.tf  vpc.tf
ubuntu@ip-172-31-37-161:~/Trend/terraform$
```

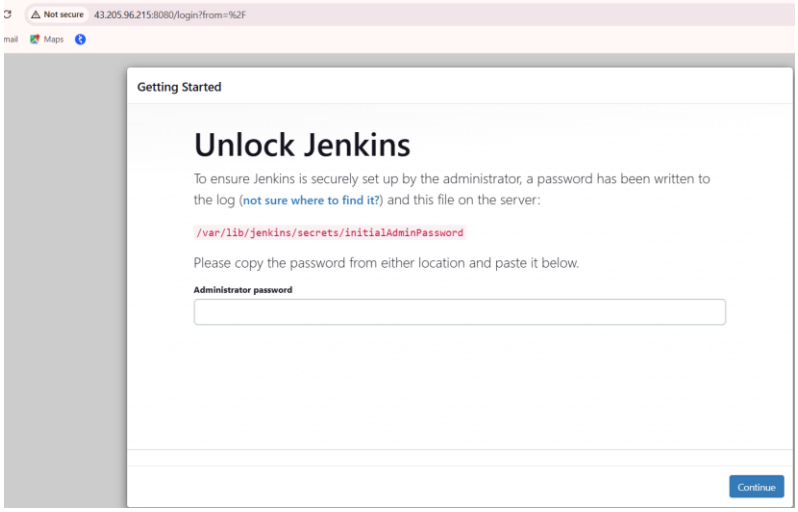
10. Apply terraform init,plan and apply

```
ubuntu@ip-172-31-37-161:~/Trend/terraform$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.41.0...
- Installed hashicorp/aws v6.41.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
ubuntu@ip-172-31-37-161:~/Trend/terraform$
```

```
      + ipv6_cidr_block_network_border_group = (known after apply)
      + main_route_table_id                  = (known after apply)
      + owner_id                             = (known after apply)
      + region                               = "ap-south-1"
      + tags_all                             = (known after apply)
    }

Plan: 8 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + jenkins_public_ip = (known after apply)
```

```
Changes to Outputs:
  + jenkins_public_ip = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_vpc.main: Creating...
aws_vpc.main: Creation complete after 2s [id=vpc-0b4a5041c050d4196]
aws_internet_gateway.igw: Creating...
aws_subnet.public: Creating...
aws_route_table.rt: Creating...
aws_security_group.jenkins_sg: Creating...
aws_internet_gateway.igw: Creation complete after 2s [id=igw-04f3b01991545b95a]
aws_route_table.rt: Creation complete after 2s [id=rtb-007a53f32c90950e3]
aws_route.r: Creating...
aws_route.r: Creation complete after 1s [id=r-rtb-007a53f32c90950e31080289494]
aws_security_group.jenkins_sg: Creation complete after 4s [id=sg-0b40acd2361bd6f
08]
aws_subnet.public: Still creating... [00m10s elapsed]
aws_subnet.public: Creation complete after 12s [id=subnet-051d1bd4b2ea2da46]
aws_route_table_association.a: Creating...
aws_instance.jenkins: Creating...
aws_route_table_association.a: Creation complete after 1s [id=rtbassoc-06f9b16ad
13533058]
aws_instance.jenkins: Still creating... [00m10s elapsed]
aws_instance.jenkins: Creation complete after 14s [id=i-0cdcf2af7bc66ce60]

Apply complete! Resources: 8 added, 0 changed, 0 destroyed.

Outputs:

jenkins_public_ip = "43.205.96.215"
```

11. Ec2 lunched and jenkin installed



12. Login to jenkin server

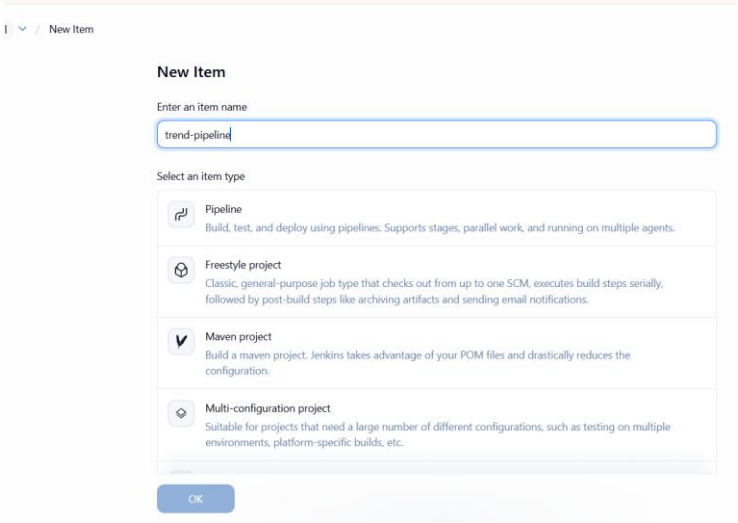
```
ubuntu@ip-10-0-1-139:~$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword
71aba6a5a3f749728d73516de4174bcf
ubuntu@ip-10-0-1-139:~$
```

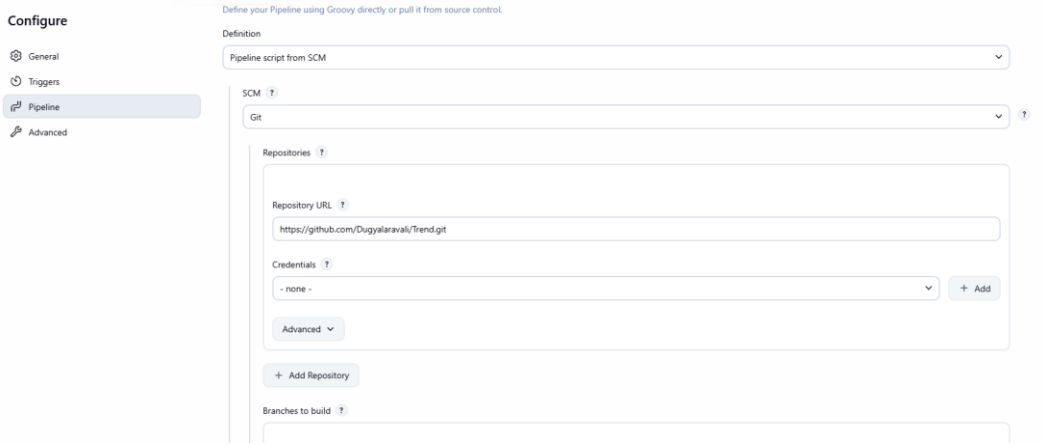
13. Writeing Jenkins file

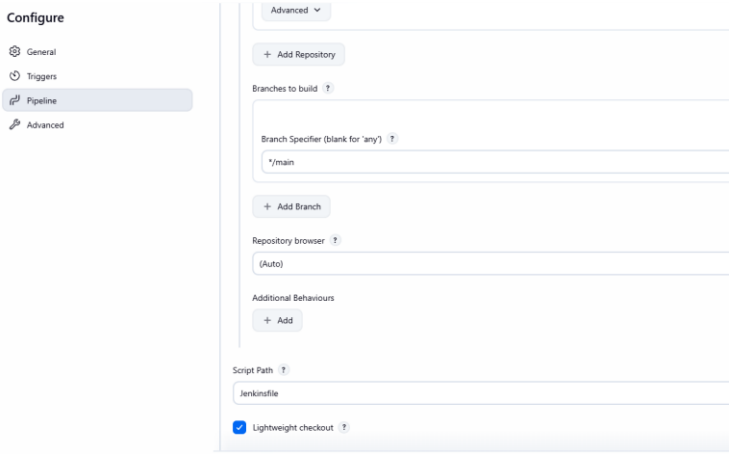
```
pipeline {
    agent any

    environment {
        DOCKER_IMAGE = "dugyalaravali28/trend-app"
        KUBE_NAMESPACE = "monitoring"
    }

    stages {

        stage('Clone Repo') {
            steps {
                git branch: 'main' , url: 'https://github.com/Dugyalaravali/Trend.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t $DOCKER_IMAGE:latest .'
            }
        }

        stage('Login to DockerHub') {
            steps {
                withCredentials([usernamePassword(
                    credentialsId: 'docker-creds',
                    usernameVariable: 'USER',
                    passwordVariable: 'PASS'
                )]) {
                    sh 'echo $PASS | docker login -u $USER --password-stdin'
                }
            }
        }

        stage('Push Image') {
            steps {
                sh 'docker push $DOCKER_IMAGE:latest'
            }
        }

        stage('Deploy App to Kubernetes') {
            steps {
                sh 'kubectl apply -f deployment.yaml'
                sh 'kubectl apply -f service.yaml'
            }
        }

        stage('Setup Monitoring (Prometheus & Grafana)') {
            steps {
                sh '''
                kubectl create namespace $KUBE_NAMESPACE || true

                helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
                helm repo update

                helm upgrade --install monitoring prometheus-community/kube-prometheus-stack \
                --namespace $KUBE_NAMESPACE
```

```
            }
        }

        stage('Verify Monitoring') {
            steps {
                sh 'kubectl get pods -n $KUBE_NAMESPACE'
                sh 'kubectl get svc -n $KUBE_NAMESPACE'
            }
        }
    }
}
```

14. Steup Jenkins cicd pipeline







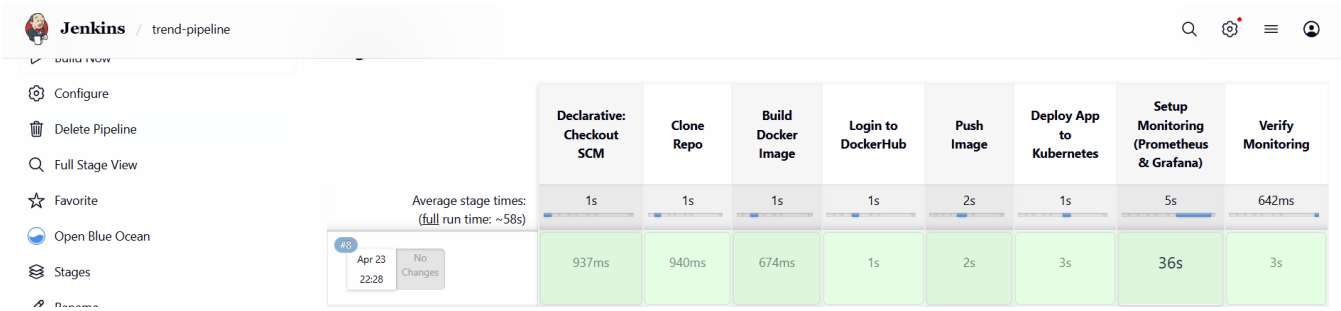
15. Jenkin server connected with eks

```
ubuntu@ip-172-31-34-24:~$ sudo ./aws/install
You can now run: /usr/local/bin/aws --version
ubuntu@ip-172-31-34-24:~$ aws --version
aws-cli/2.34.30 Python/3.14.4 Linux/6.17.0-1007-aws exe/x86_64.ubuntu.24
ubuntu@ip-172-31-34-24:~$ aws eks --region ap-south-1 update-kubeconfig --n
ame my-cluster
Added new context arn:aws:eks:ap-south-1:736956442736:cluster/my-cluster to
 /home/ubuntu/.kube/config
ubuntu@ip-172-31-34-24:~$ docker --version
kubectl version --client
kubectl get nodes
Docker version 28.4.0, build d8eb465
Client Version: v1.35.4
Kustomize Version: v5.7.1
NAME                                          STATUS   ROLES    AGE   VER
SION
ip-192-168-25-187.ap-south-1.compute.internal   Ready    <none>   82m   v1.
34.6-eks-bbe087e
ip-192-168-54-134.ap-south-1.compute.internal   Ready    <none>   52m   v1.
34.6-eks-bbe087e
ip-192-168-64-105.ap-south-1.compute.internal   Ready    <none>   82m   v1.
34.6-eks-bbe087e
ubuntu@ip-172-31-34-24:~$ sudo usermod -aG docker jenkins
sudo chmod 666 /var/run/docker.sock
sudo systemctl restart jenkins
```

16. Installed helm Jenkins server

```
ubuntu@ip-172-31-40-189:~$ curl https://raw.githubusercontent.com/helm/helm/main
/scripts/get-helm-3 | bash
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 11929  100 11929    0     0   276k      0 --:--:-- --:--:-- --:--:--  277k
Downloading https://get.helm.sh/helm-v3.20.2-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
helm installed into /usr/local/bin/helm
ubuntu@ip-172-31-40-189:~$ helm version
version.BuildInfo{Version:"v3.20.2", GitCommit:"8fb76d6ab555577e98e23b7500009537
a471feee", GitTreeState:"clean", GoVersion:"go1.25.9"}
ubuntu@ip-172-31-40-189:~$ |
```

17. Pipeline succefully deployed on eks



18. Pipeline logs

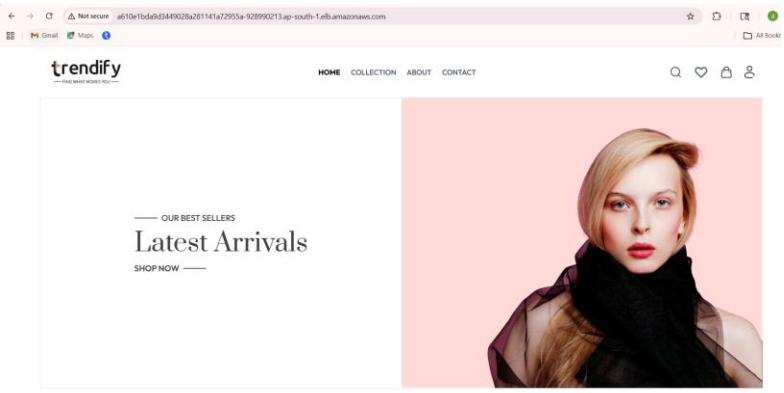
```
Visit https://github.com/prometheus-operator/kube-prometheus for instructions on how to create & configure Alertmanager and Prometheus instances using the Operator.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Verify Monitoring)
[Pipeline] sh
+ kubectl get pods -n monitoring
NAME                                                     READY   STATUS              RESTARTS   AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0   0/2     PodInitializing     0          5s
monitoring-grafana-85ddbd7578-x7qg7                      0/3     ContainerCreating   0          12s
monitoring-kube-prometheus-operator-7447cc9b9c-n8j92     1/1     Running             0          12s
monitoring-kube-state-metrics-5b8ff7679f-wqkqk           0/1     Running             0          12s
monitoring-prometheus-node-exporter-819hx                1/1     Running             0          12s
monitoring-prometheus-node-exporter-f2gj8                1/1     Running             0          12s
monitoring-prometheus-node-exporter-kpmrt                1/1     Running             0          12s
monitoring-prometheus-node-exporter-vdt72                1/1     Running             0          12s
prometheus-monitoring-kube-prometheus-prometheus-0       0/2     PodInitializing     0          5s
[Pipeline] sh
+ kubectl get svc -n monitoring
NAME                                      TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)                      AGE
alertmanager-operated                     ClusterIP   None             <none>        9093/TCP,9094/TCP,9094/UDP   7s
monitoring-grafana                        ClusterIP   10.100.248.214   <none>        80/TCP                       14s
monitoring-kube-prometheus-alertmanager   ClusterIP   10.100.148.171   <none>        9093/TCP,8080/TCP            14s
monitoring-kube-prometheus-operator       ClusterIP   10.100.92.163    <none>        443/TCP                      14s
monitoring-kube-prometheus-prometheus     ClusterIP   10.100.248.3     <none>        9090/TCP,8080/TCP            14s
monitoring-kube-state-metrics             ClusterIP   10.100.230.245   <none>        8080/TCP                     14s
monitoring-prometheus-node-exporter       ClusterIP   10.100.126.34    <none>        9100/TCP                     14s
prometheus-operated                       ClusterIP   None             <none>        9090/TCP                     7s
[Pipeline] }
```

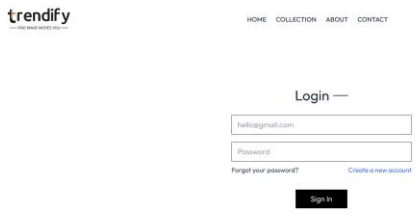
```
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
[Checks API] No suitable checks publisher found.
Finished: SUCCESS
```

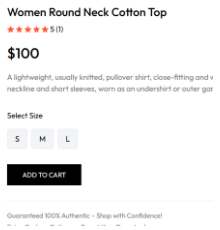
19. External ip for application.

```
NAME             TYPE           CLUSTER-IP       EXTERNAL-IP
                                                 PORT(S)        AGE
kubernetes       ClusterIP      10.100.0.1       <none>
                                                 443/TCP        3h1m
trend-service    LoadBalancer   10.100.63.211    a610e1bda9d3449028a281141a7295
5a-928990213.ap-south-1.elb.amazonaws.com        80:30000/TCP   119m
jenkins@ip-172-31-34-24:~$
```

20. Open on the browser.







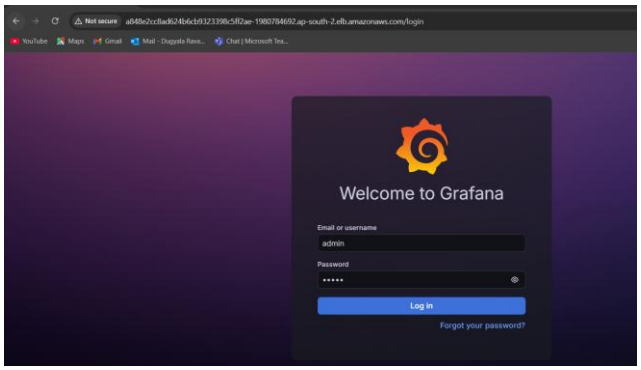
21. Checking monitoring pods

```
ubuntu@ip-172-31-40-189:~$ kubectl get pods -n monitoring
NAME                                                     READY   STATUS    RESTA
RTS   AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0   2/2     Running   0
      31s
monitoring-grafana-85ddbd7578-x7qg7                      2/3     Running   0
      38s
monitoring-kube-prometheus-operator-7447cc9b9c-n8j92     1/1     Running   0
      38s
monitoring-kube-state-metrics-5b8ff7679f-wqkqk           1/1     Running   0
      38s
monitoring-prometheus-node-exporter-8l9hx                1/1     Running   0
      38s
monitoring-prometheus-node-exporter-f2gj8                1/1     Running   0
      38s
monitoring-prometheus-node-exporter-kpmrt                1/1     Running   0
      38s
monitoring-prometheus-node-exporter-vdt72                1/1     Running   0
      38s
prometheus-monitoring-kube-prometheus-prometheus-0       2/2     Running   0
      31s
ubuntu@ip-172-31-40-189:~$
```

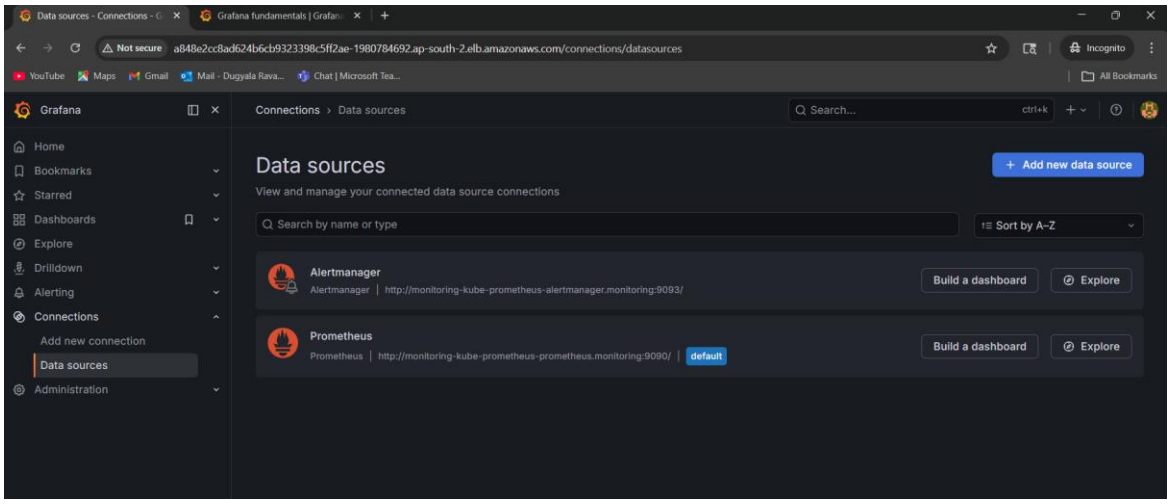
22. External ip for monitoring

```
ubuntu@ip-172-31-40-189:~$ kubectl get svc -n monitoring
NAME                                      TYPE           CLUSTER-IP       EXTERN
AL-IP                                                                    PORT(S)
                  AGE
alertmanager-operated                     ClusterIP      None             <none>
                                                                         9093/TCP,90
94/TCP,9094/UDP   113m
monitoring-grafana                        LoadBalancer   10.100.147.18    a848e2
cc8ad624b6cb9323398c5ff2ae-1980784692.ap-south-2.elb.amazonaws.com   80:30625/TC
P                 94m
monitoring-kube-prometheus-alertmanager   ClusterIP      10.100.148.171   <none>
                                                                         9093/TCP,80
80/TCP            113m
monitoring-kube-prometheus-operator       ClusterIP      10.100.92.163    <none>
                                                                         443/TCP
                  113m
monitoring-kube-prometheus-prometheus     ClusterIP      10.100.248.3     <none>
                                                                         9090/TCP,80
80/TCP            113m
monitoring-kube-state-metrics             ClusterIP      10.100.230.245   <none>
                                                                         8080/TCP
                  113m
monitoring-prometheus-node-exporter       ClusterIP      10.100.126.34    <none>
                                                                         9100/TCP
                  113m
prometheus-operated                       ClusterIP      None             <none>
                                                                         9090/TCP
                  113m
ubuntu@ip-172-31-40-189:~$
```

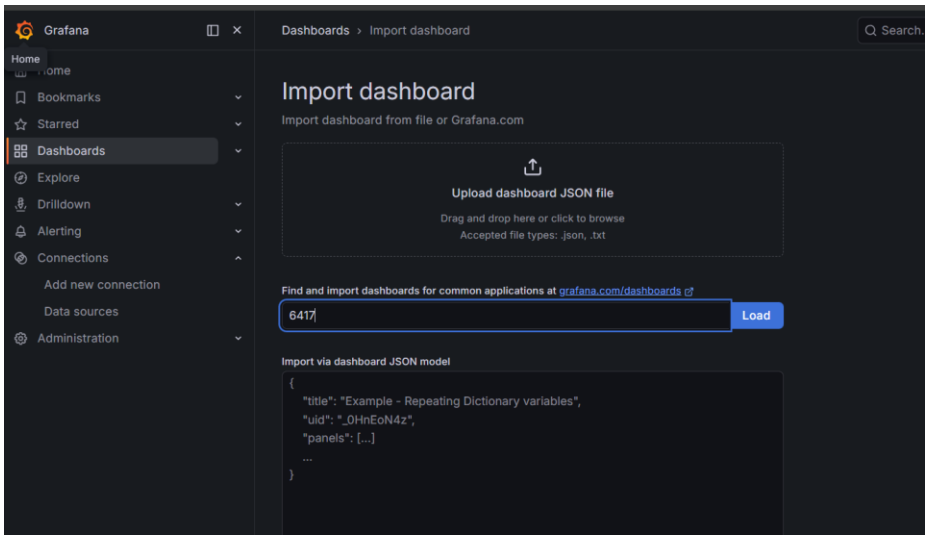
23. Open in browser.

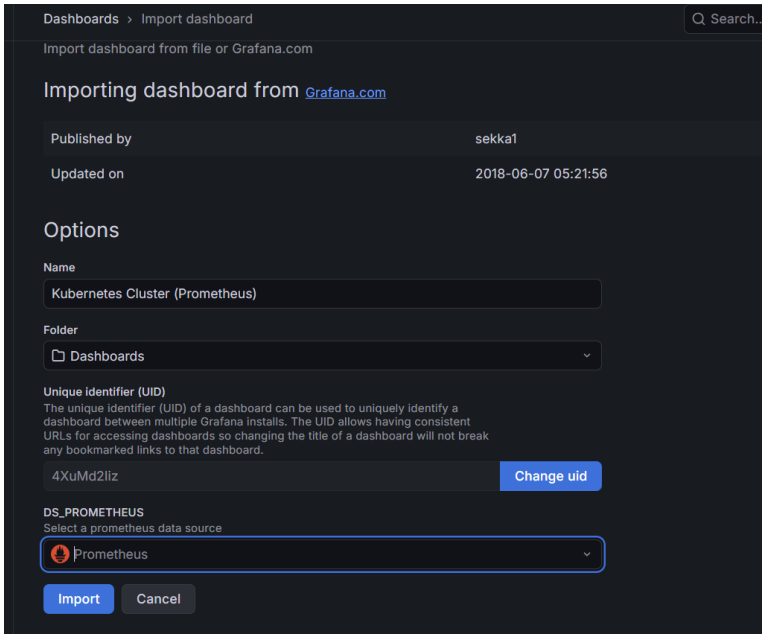


24. Added data source.

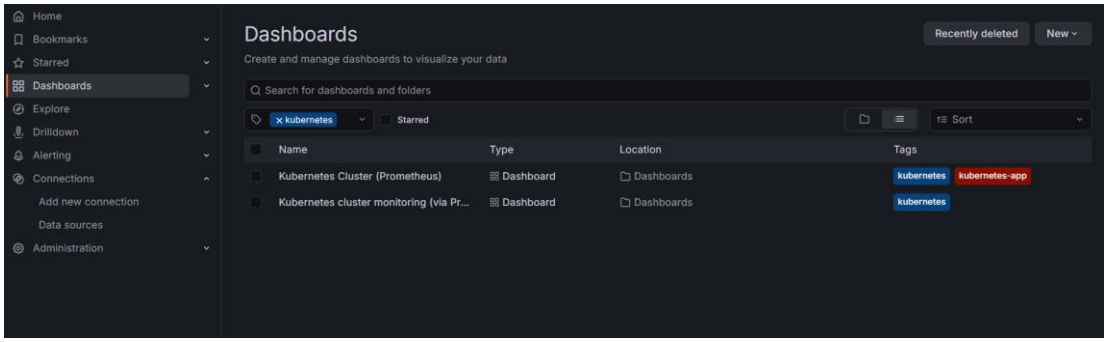


25. import dashboards.

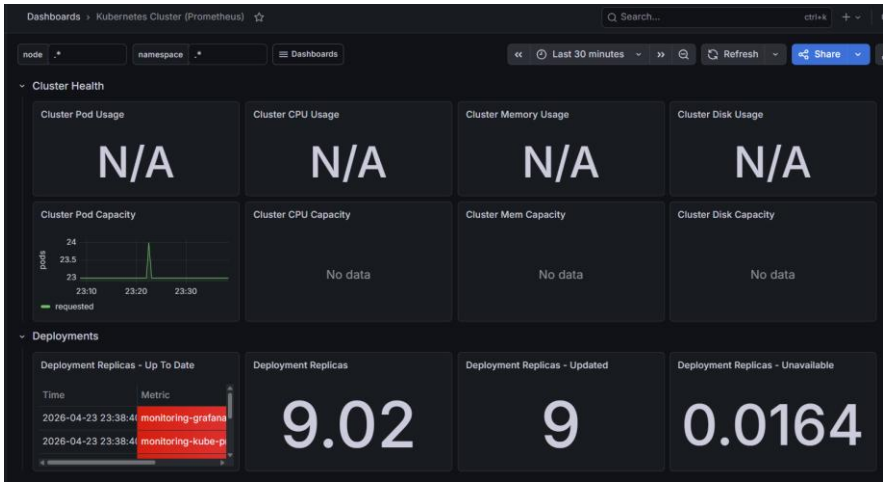


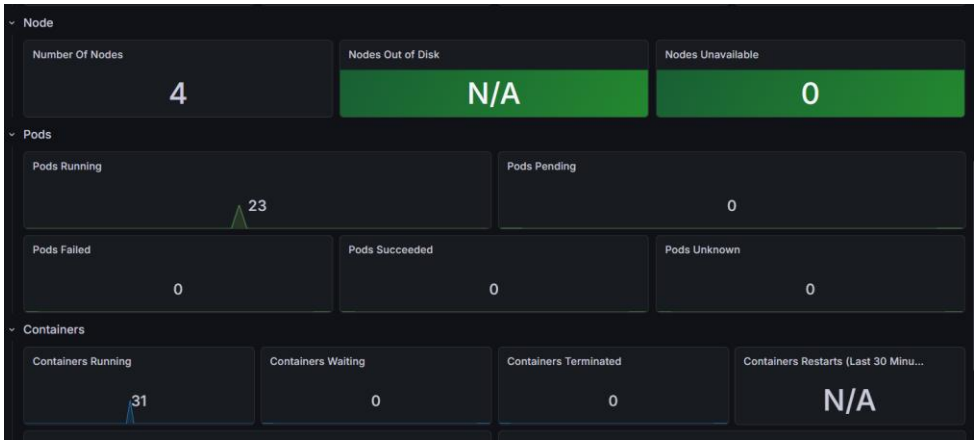


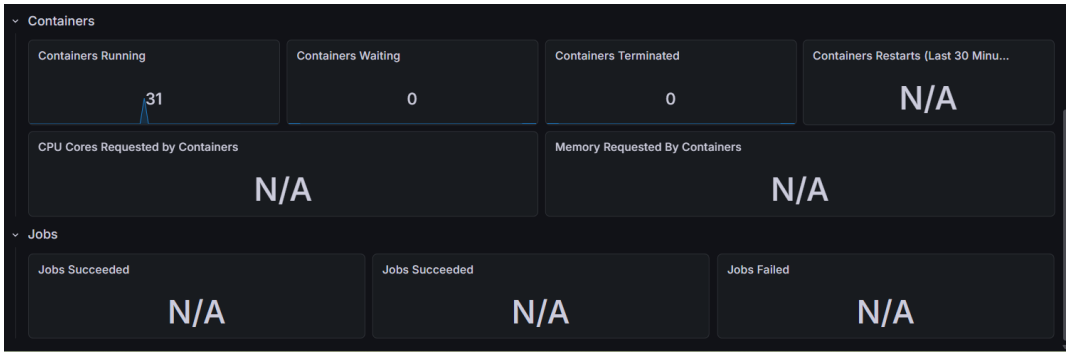
26. checking dashboards.



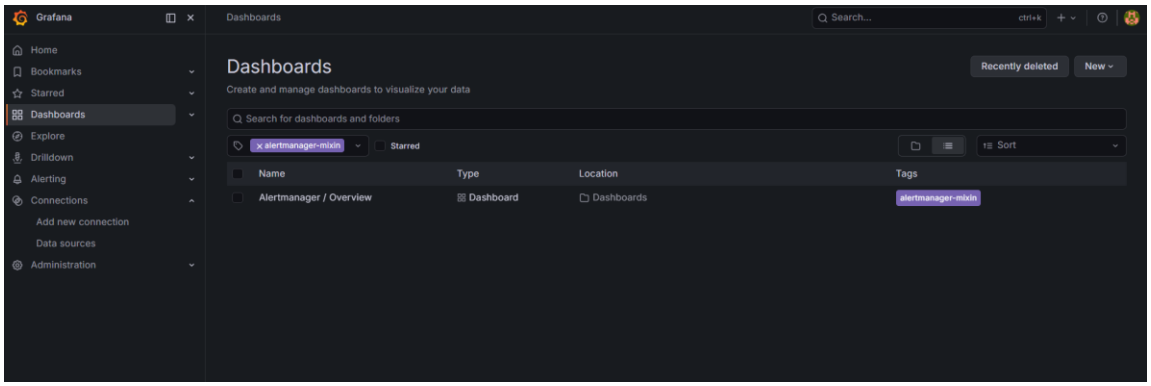
27. seeing cpu,nodes,pods data.



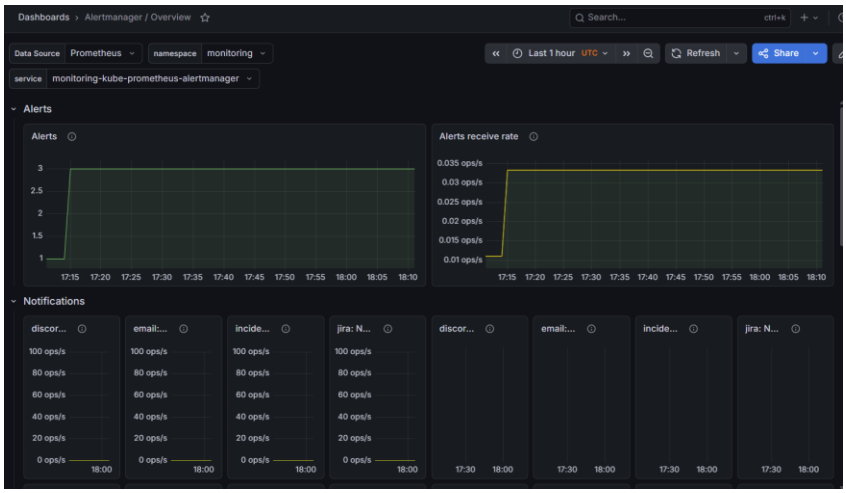


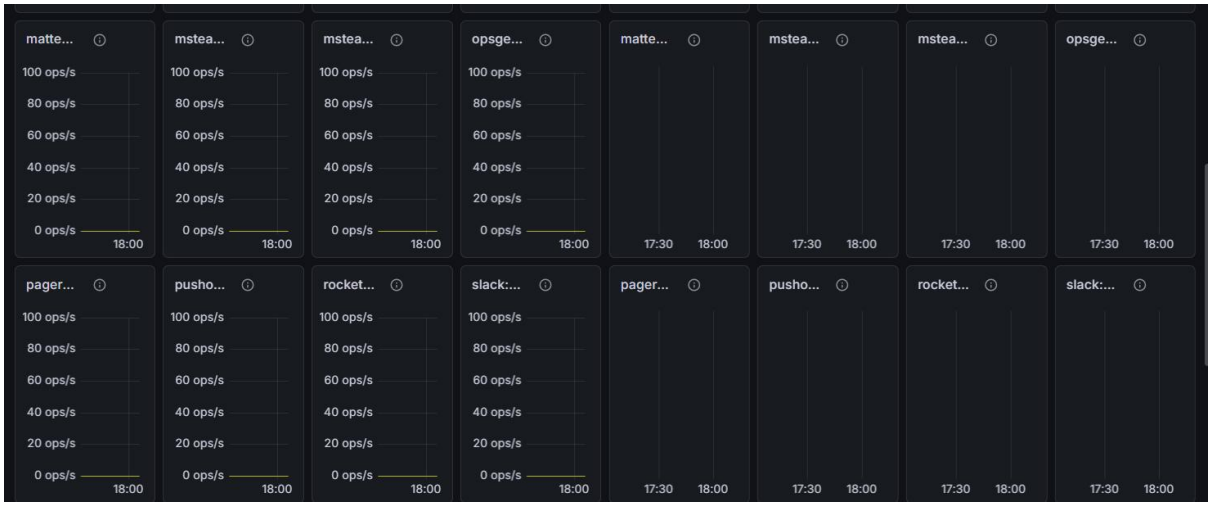


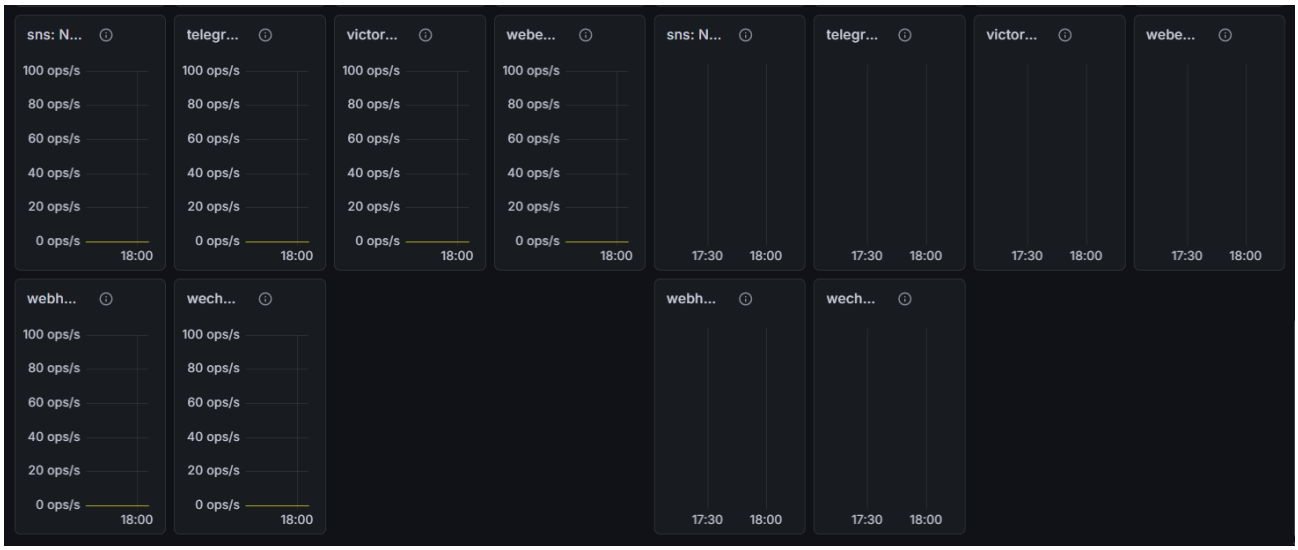
28. checking alerting dashboard.



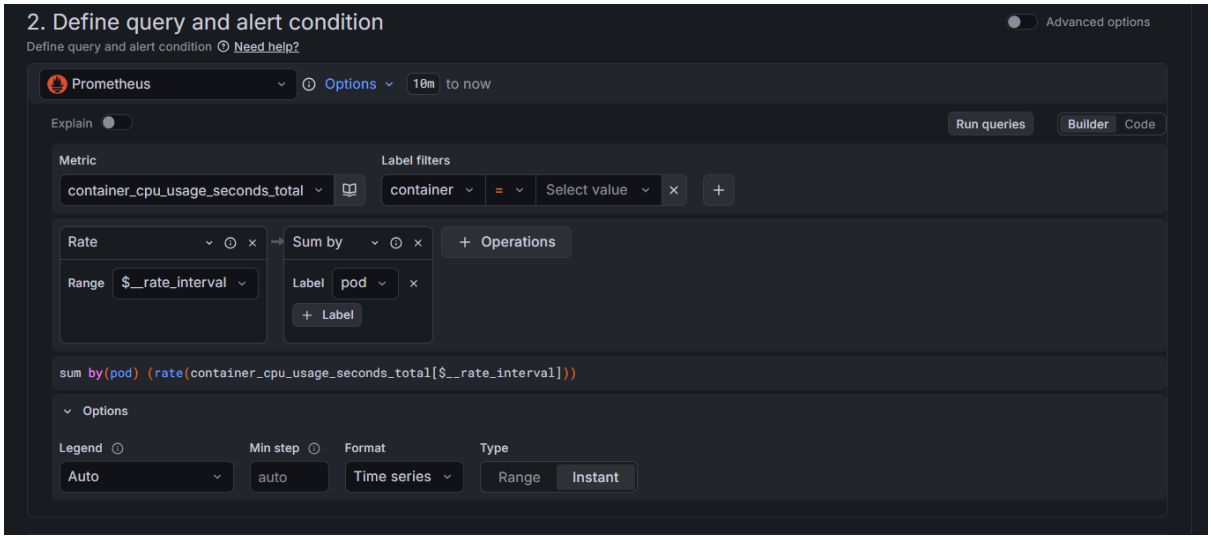
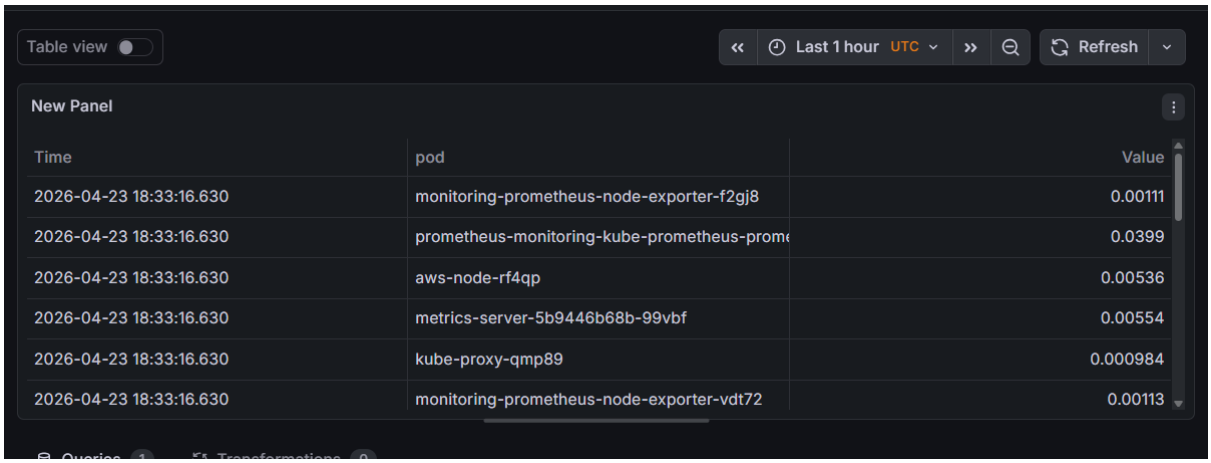
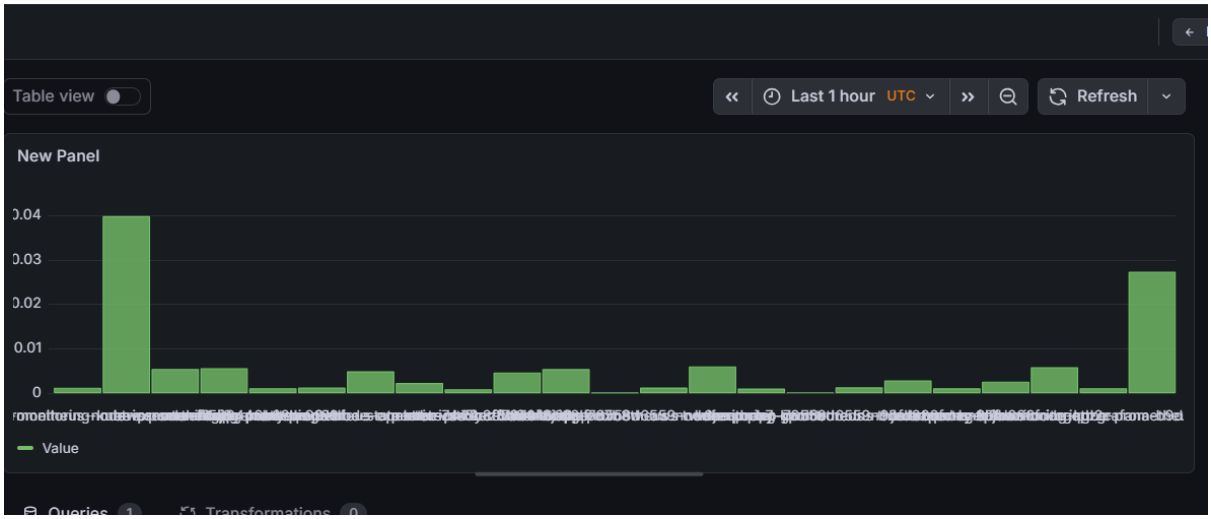
29. data of alerts dashboard.







30. create alert for cpu for pod



31. run the query and see the cpu usage per pod in table,graph and bar modules.